



Universidade Estadual de Santa Cruz
Programa de Pós Graduação em Modelagem Computacional - PPGMC

PROJETO 1 DE MÉTODOS NUMÉRICOS I

Alunos: Igor Sousa dos Santos Santana e
Matheus Santos Silva
Professor: Dany Sanches Domingues

Ilhéus
02/06/2025

RESUMO

Este projeto tem como objetivo aplicar e comparar diferentes métodos numéricos para a solução de sistemas de equações lineares e cálculo de matrizes inversas, no contexto da disciplina de Métodos Numéricos I. No primeiro exercício, foram avaliados métodos diretos (Eliminação de Gauss em suas variações e fatoração LU) e métodos iterativos (Jacobi, Gauss-Seidel e Relaxamento com Sobre-Relaxamento e Sub-Relaxamento) na resolução de sistemas lineares utilizando matrizes de Hilbert de ordens 5, 9 e 15. A análise envolveu critérios como precisão, tempo de execução, convergência e sensibilidade ao condicionamento das matrizes. Os métodos diretos apresentaram maior precisão, enquanto os iterativos mostraram limitações em termos de convergência, especialmente para matrizes mal condicionadas. No segundo exercício, o método de fatoração LU foi utilizado para cálculo de inversas de duas matrizes distintas. Verificou-se, por meio do determinante e da triangularização, que apenas uma das matrizes era invertível. Os resultados obtidos foram validados por meio da multiplicação da matriz original por sua inversa, evidenciando boa precisão nos cálculos. O projeto reforça a importância da escolha adequada do método numérico conforme o problema em questão, considerando fatores como condicionamento da matriz, complexidade computacional e objetivo da análise.

SUMÁRIO

PROJETO 1 DE MÉTODOS NUMÉRICOS I.....	1
RESUMO.....	2
SUMÁRIO.....	2
1. INTRODUÇÃO.....	4
2. MATERIAIS.....	5
3. METODOLOGIA.....	8
4. RESULTADOS E DISCUSSÃO.....	12
5. CONCLUSÃO.....	31
6. APÊNDICE.....	32
6.1. Link para projeto no GitHub.....	32
6.2. Estrutura de pastas e arquivos.....	32
6.3. Instalando as dependências do projeto.....	34

1. INTRODUÇÃO

O presente relatório documenta o desenvolvimento do projeto 1 da disciplina Métodos Numéricos I. O conteúdo do projeto trata-se da resolução de dois exercícios propostos pelo docente sobre o tema: métodos de solução de sistemas de equações lineares e algébricas.

No exercício 1, é proposto a resolução de sistemas lineares a partir da matriz de Hilbert de tamanho n , utilizando-se dos seguintes métodos: Eliminação de Gauss com substituição regressiva; Eliminação de Gauss com pivoteamento parcial; Eliminação de Gauss com pivoteamento parcial com escala; Eliminação de Gauss com pivoteamento total; Jacobi; Gauss-Seidel; Relaxamento com a estratégia do sobre-relaxamento. Além disso, deve-se calcular o erro verdadeiro da solução para cada um dos métodos e comparar os métodos em relação a precisão, convergência e desempenho computacional.

No exercício 2, é proposto a utilização do método de fatoração LU , sem pivoteamento, para calcular as inversas das matrizes. E após o cálculo das inversas, deve ser feita a verificação do resultado, fazendo $[A] \cdot [A]^{-1} = [I]$.

Neste documento serão abordados os materiais utilizados, a metodologia, os resultados e discussões e a conclusão.

O próximo capítulo irá apresentar os materiais utilizados para implementação e validação dos métodos, realização dos testes e criação dos gráficos e tabelas.

2. MATERIAIS

Este capítulo descreve os materiais utilizados para a implementação dos modelos matemáticos utilizados ao longo do trabalho.

As ferramentas e pacotes utilizados para implementação dos modelos foram:

- Linguagem de programação *Python* em sua versão 3.13;
- Anaconda software para criação e manutenção de ambientes virtuais Python.
- Biblioteca *matplotlib* em sua versão 3.10.1, para a geração de gráficos;
- Biblioteca *NumPy* em sua versão 2.2.0, para a implementação dos modelos matemáticos;
- Biblioteca *pandas* em sua versão 2.2.3, para a criação de *DataFrames*;
- Biblioteca *sciPy* em sua versão 1.15.3, para a realização de cálculos complexos;
- Jupyter Notebook para a criação dos modelos numéricos.

Para a resolução dos exercícios, foram utilizados os métodos de sistema de equações lineares e algébricas (SELA): Eliminação de Gauss (sem pivoteamento, com pivoteamento parcial, pivoteamento parcial com escalas e pivoteamento completo), fatoração *LU*, método de Jacobi, método de Gauss-Seidel e o método do Relaxamento (com a estratégia do sobre-relaxamento).

A eliminação de Gauss é um algoritmo direto para resolver sistemas de equações lineares $Ax = b$, onde A é uma matriz de coeficientes $n \times n$ (real ou complexa), x é o vetor de incógnitas e b é o vetor de termos independentes. O método procede em duas fases: a eliminação direta, transformando A em uma matriz triangular superior, e a substituição regressiva. Neste caso, não há pivotamento.

Para a eliminação de Gauss com pivotamento parcial, os passos são os mesmo do eliminação de Gauss simples, mas antes de cada etapa de eliminação k , a coluna k é escaneada verticalmente a partir da linha k . O elemento com o maior valor absoluto na coluna k , entre as linhas k e n , é identificado. A linha que contém este elemento é então trocada com a linha k . O elemento $a_{kk}^{(k)}$ resultante após a troca de linha é usado como pivô.

Para a eliminação de Gauss com pivoteamento parcial com escala, os passos são os mesmos da eliminação de Gauss com pivoteamento parcial, mas incorpora uma consideração de escala. Antes da eliminação, um vetor de fatores de escala $S = [s_1, s_2, \dots, s_n]$ é calculado,

onde $S_i = \max_j |a_{ij}|$. Em cada etapa k , ao invés de buscar o maior $|a_{ij}^{(k)}|$, busca-se o índice p que maximiza a razão $\frac{|a_{kp}^{(k)}|}{S_p}$ para $p \geq k$. A linha p então é trocada com a linha k .

Para o caso da eliminação Gauss com pivotamento completo, o algoritmo segue a mesma lógica da eliminação de Gauss com pivotamento parcial com escalas, mas na etapa k , busca-se o elemento com o maior valor absoluto em toda uma submatriz restante não processada nas linhas k a n e colunas k a n . Se este elemento tiver o maior valor absoluto, a linha p é trocada com a linha k , e a coluna q é trocada com a coluna k . Isso implica que a ordem das variáveis em x também muda e também é necessário uma forma de rastrear essas permutações de coluna.

A fatoração LU segue o seguinte princípio: decompor a matriz de coeficientes A em um produto de duas matrizes triangulares: uma matriz triangular inferior L com elementos 1 na diagonal principal, e uma matriz triangular superior U . Para decomposição da matriz A , o algoritmo da fatoração LU utiliza uma variante da eliminação de Gauss. Os multiplicadores m_{ik} utilizados na eliminação, para zerar os elementos abaixo da diagonal, são armazenados na matriz L nas posições $L_{ik} = m_{ik}$. Os elementos da matriz U são os elementos resultantes da matriz A após a eliminação.

Os métodos de Jacobi, Gauss-Seidel e Relaxamento partem de uma estimativa inicial x_0 da solução e geram uma sequência de aproximações $x_k, x_{(k+1)}, \dots$ que converge para a solução exata x se algumas condições forem satisfeitas.

O método de Jacobi isola a variável x_i na i -ésima equação e utiliza os valores das iterações anteriores das outras variáveis para gerar novas aproximações. O método de Jacobi Converge se a matriz A for estritamente diagonalmente dominante por linhas ou colunas. Porém não há garantia que ela vai divergir para outros casos.

O método de Gauss-Seidel é uma melhoria do método de Jacobi. Ele utiliza imediatamente os valores das variáveis que já foram calculadas na iteração atual para $x_j^{(k+1)}$ onde $j < i$ à medida que avança na determinação de $x_i^{(k+1)}$. O método de Gauss-Seidel possui o mesmo critério de convergência que o método de Jacobi.

O método do Relaxamento é uma extensão do método de Gauss-Seidel que introduz um parâmetro de relaxamento ω para acelerar ou desacelerar a convergência. Neste trabalho foi utilizada a estratégia do sobre-relaxamento, onde $1 < \omega < 2$, que é utilizado para

acelerar a convergência. Mais precisamente, neste trabalho, foi utilizado: $\omega = 1, 2$. O método do Relaxamento possui o mesmo critério de convergência que o método de Gauss-Seidel.

Para a resolução do exercício 1, foi utilizada a matriz de Hilbert, que é construída da seguinte forma demonstrada na Equação 1.

$$h_{ij} = \frac{1}{i+j-1} \quad (1)$$

Onde:

$$i = 1:n$$

$$j = 1:n$$

A resolução deve ser feita para os sistemas apresentados pela Equação 2.

$$Hx = b \quad (2)$$

Onde os elementos do vetor independente b_i são calculados como a soma de todos os elementos da linha i da matriz de Hilbert como demonstrado na Equação 3.

$$b_i = \sum_{j=1}^n h_{ij} \quad (3)$$

Foram utilizadas as matrizes de Hilbert com $n = 5$, $n = 9$ e $n = 15$.

Para a resolução do exercício 2, foram utilizadas as matrizes abaixo:

$$A_1 = \begin{bmatrix} 10 & 2 & -1 \\ -3 & -6 & 2 \\ 1 & 1 & 5 \end{bmatrix}$$

$$A_2 = \begin{array}{|cccc|} \hline 1 & 4 & 9 & 16 \\ 4 & 9 & 16 & 25 \\ 9 & 16 & 25 & 36 \\ 16 & 25 & 36 & 49 \\ \hline \end{array}$$

No próximo capítulo será abordada a metodologia e as estratégias aplicadas para a resolução dos exercícios propostos.

3. METODOLOGIA

Neste capítulo será abordada a metodologia utilizada durante o desenvolvimento deste trabalho, abordando os passos para a implementação dos métodos, para a resolução dos exercícios propostos e para avaliação da precisão, desempenho computacional e comportamento de convergência dos métodos utilizados.

A implementação dos métodos foi realizada seguindo os pseudo-algoritmos fornecidos na disciplina de métodos numéricos. Nessas implementações foram realizadas pequenas alterações para garantir uma melhor modularização dos métodos. Nos métodos diretos, os passos para eliminação de gauss e substituição foram transformados em funções. Essa abordagem permite a reutilização desses métodos em outros métodos implementados. Para os métodos iterativos, foi desenvolvida uma função base que realiza os passos iterativos e que tem como parâmetros adicionais a fórmula iterativa e o nome do método. A figura a seguir ilustra a implementação da função base para os métodos iterativos.

Figura 1 - Função base para os métodos iterativos.

Função Base (matriz aumentada, tolerância, número de iterações, fórmula iterativa, nome do método)

```
PASSO 1: Para i=1:n faça passo 2
  PASSO 2:  $x_{0_i} = A_{in}/A_{ii}$ 
PASSO 3: Faça k=1
PASSO 4: Enquanto (k<N) faça passos 5-8
  PASSO 5: Para i=1:n faça  $x_i =$  fórmula iterativa
  PASSO 6: Se  $||x-x_0|| < \text{tol}$  Saída ( $x_1, \dots, x_n$ ), FIM
  PASSO 7: k=k+1
  PASSO 8: Para i=1:n faça  $x_{0_i} = x_i$ 
PASSO 9: Saída (Método Falhou), FIM
```

A implementação dos métodos de Jacobi, Gauss-Seidel e Relaxamento (SOR) consistiu na criação de funções específicas para cada uma de suas respectivas fórmulas de iteração. Essas funções foram, então, passadas como argumento para uma rotina base comum, responsável por gerenciar o processo iterativo.

Para os métodos iterativos, a escolha do vetor de aproximação inicial, $x^{(0)}$, seguiu o critério definido pela Equação (8), onde cada componente x_i^0 é dado por:

$$x^{(0)} = \frac{b_i}{a_{ii}} \quad (8)$$

Onde:

$x^{(0)}$ é a aproximação inicial que desejamos obter.

b_i é um elemento do vetor de resultados na posição i .

a_{ii} é um elemento da diagonal da matriz A .

Além dos métodos mencionados, foi desenvolvida uma função para a solução de sistemas de equações lineares utilizando a fatoração LU . Para isso, também foi implementada uma função auxiliar responsável pela substituição progressiva. O algoritmo adotado segue as etapas clássicas: fatoração da matriz A nas matrizes L e U , resolução do sistema triangular inferior $L \cdot y = b$ por substituição progressiva, e posterior resolução do sistema triangular superior $U \cdot x = y$ por substituição regressiva. A função de solução via fatoração LU também

foi empregada no cálculo das matrizes inversas, conforme exigido na resolução do exercício 2.

A validação dos métodos implementados foi realizada com base nas matrizes fornecidas nos materiais de apoio.

A seguir, são descritos os procedimentos adotados para a resolução dos Exercícios 1 e 2.

Para a resolução do exercício 1, inicialmente foram geradas matrizes de Hilbert de ordens $n = 5$, $n = 9$ e $n = 15$, utilizando a fórmula apresentada na equação 1.

Posteriormente, para cada uma das três matrizes geradas, realizou-se a análise de seu condicionamento. Essa análise incluiu a verificação da singularidade por meio do determinante e o cálculo do número de condicionamento para quantificar a sensibilidade da matriz. O condicionamento de uma matriz indica a sensibilidade da solução de um sistema linear $Ax = b$ a perturbações nos dados de entrada: matrizes bem condicionadas apresentam pequena variação na solução para pequenas variações na entrada, enquanto matrizes mal condicionadas podem apresentar grandes variações na solução.

Para avaliação do condicionamento das matrizes, foram considerados o determinante e o número de condicionamento. O determinante de cada matriz foi calculado utilizando a função `linalg.det(A)` do pacote `NumPy`. O número de condicionamento foi calculado utilizando a função `linalg.cond(A, p = np.inf)` do `NumPy`, que internamente emprega a norma máxima (norma- ∞).

A definição do número de condicionamento utilizando a norma- ∞ é:

$$\text{cond}(A) = \|A\| \cdot \|A^{-1}\| \quad (4)$$

Valores de determinante próximos a zero e números de condicionamento significativamente maiores que 1 serão considerados indicativos de mau condicionamento nesta análise.

Como último teste, foi verificado se as matrizes eram diagonalmente dominante por linhas para verificar se poderíamos afirmar se os métodos de Jacobi e Gauss-Seidel convergem para a matriz de Hilbert. Para uma matriz ser diagonal dominante, deve verificar se, para cada linha, o valor absoluto do elemento da diagonal principal for maior ou igual à soma dos valores absolutos de todos os outros elementos, não diagonais, naquela linha.

A fórmula para verificar se uma matriz diagonal por linhas é dada por:

$$|a_{ii}| \geq \sum_{j \neq i} |a_{ij}|, \text{ para todo } i = 1, 2, \dots, n$$

Onde:

a_{ii} é o i-ésimo elemento da diagonal principal.

a_{ij} são os outros elementos na i-ésima linha (onde $j \neq i$).

Após realizar os testes nas matrizes, foram executados os algoritmos para os métodos. Foram testados primeiramente os métodos diretos e logo após os métodos iterativos. Para a execução de cada método foi calculado o tempo de execução para determinar o desempenho de cada método. O tempo de execução foi calculado como a diferença entre o tempo inicial e o tempo final. Foi utilizado a função *perf_counter* da biblioteca *time* para o cálculo do tempo. Além disso, foi verificado a convergência dos métodos e sua precisão.

Para verificar a convergência, foram utilizadas as tolerâncias 10^{-3} e 10^{-6} , e número máximo de iterações igual a 1 milhão (1×10^7). Além disso, foi registrado o comportamento do erro do modelo durante as iterações.

Para verificar a precisão do modelo, foi utilizado a norma máxima entre o valor real e o valor de solução calculado pelos métodos. Para o cálculo da norma máxima foi utilizada a função *linalg.norm* do *NumPy*.

O valor da norma máxima é obtido através da resolução da Equação 5 a seguir.

$$\|x - \hat{x}\|_{\infty} = \max_{1 \leq i \leq n} \left| \frac{x_i - \hat{x}_i}{x_i} \right| \quad (5)$$

Onde:

x_i é o i-ésimo componente do vetor exato.

$\hat{x}_i$ é o i-ésimo componente do vetor aproximado.

Para a resolução do Exercício 2, inicialmente foi verificado se as matrizes eram invertíveis. Essa verificação foi feita por meio da análise da forma escalonada das matrizes, obtida a partir da matriz *U* gerada pela fatoração *LU*. Avaliou-se a presença de linhas nulas e o valor do determinante, sendo considerados como indicativos de não invertibilidade valores nulos ou próximos de zero.

Em seguida, foi gerada a matriz identidade correspondente a cada matriz analisada. A partir disso, a inversa de cada matriz foi calculada de forma iterativa, resolvendo sucessivamente sistemas lineares via fatoração LU . Cada coluna da matriz inversa foi obtida resolvendo o sistema:

$$A \cdot A_j^{(-1)} = I_j, \quad \text{para } 1 < j < n \quad (6)$$

Onde $A_j^{(-1)}$ representa a j -ésima coluna da matriz inversa e I_j é a j -ésima coluna da matriz identidade.

A resolução dos sistemas foi realizada utilizando a função desenvolvida, anteriormente comentada, para resolução de sistemas de equações lineares por meio da fatoração LU .

Por fim, foi verificada a solução fazendo $A \cdot A^{(-1)} = I$.

No próximo capítulo serão abordados os resultados retornados pelos modelos utilizados nos dois exercícios e as discussões sobre os resultados.

4. RESULTADOS E DISCUSSÃO

Nesta seção, são apresentados os resultados dos testes de condicionamento e invertibilidade das matrizes, bem como a resolução dos exercícios propostos. Também são discutidos os resultados obtidos na avaliação da precisão, do desempenho computacional e do comportamento de convergência dos métodos utilizados.

A Tabela 1 apresenta os resultados do teste de condicionamento das matrizes de Hilbert de ordens 5, 9 e 15.

Tabela 1 - Resultados do teste de condicionamento da matriz de Hilbert de ordem 5, 9 e 15.

Ordem da Matriz (n)	Determinante (det(A))	Número de Condicionamento (cond $_{\infty}$ (A))
5	$3,7493 \times 10^{-12}$	$9,4366 \times 10^5$
9	$9,7203 \times 10^{-43}$	$1,0996 \times 10^{12}$
15	$-1,2321 \times 10^{-120}$	$1,0975 \times 10^{18}$

Analisando os dados da Tabela 1, observa-se que a matriz de Hilbert possui mau condicionamento para todas as ordens testadas. Os valores do determinante encontrados nos testes são extremamente pequenos (muito próximos de zero), o que é um forte indicativo do severo mau condicionamento dessas matrizes e de sua proximidade à singularidade numérica. Esses determinantes tornam-se progressivamente menores (ainda mais próximos de zero) para as matrizes de ordem mais alta, o que demonstra que o condicionamento das matrizes de Hilbert piora à medida que sua ordem aumenta.

Já para o número de condicionamento, o resultado apresentou comportamento análogo: uma piora no condicionamento para ordens maiores, resultando em valores cada vez mais distantes do valor unitário (1), valor que, idealmente, caracteriza um bom condicionamento. Esse crescimento nos valores reforça a conclusão de que as matrizes de Hilbert tornam-se progressivamente mais mal condicionadas à medida que sua dimensão aumenta.

Esses resultados do teste de condicionamento, apresentados na Tabela 1, demonstram que as matrizes estão mal condicionadas, o que pode ocasionar que os algoritmos encontrem soluções com alto valor de erro ou, no caso de métodos iterativos, que não apresentem convergência para a solução.

A Tabela 2, a seguir, apresenta os resultados da verificação da condição de dominância diagonal por linhas para as matrizes de Hilbert de ordens 5, 9 e 15.

Tabela 2 - Resultados da verificação da condição de dominância diagonal por linhas para as matrizes de Hilbert de ordens 5, 9 e 15.

Ordem da Matriz (n)	$ a_{11} $	$\sum_{j \neq 1} a_{1j} $	Condição Satisfeita?
5	1	1,28333	Não
9	1	1,82897	Não
15	1	2,31823	Não

Ao verificar a condição de dominância diagonal por linhas para as matrizes de Hilbert de ordens 5, 9 e 15, conforme Tabela 2, constatou-se que nenhuma delas é diagonalmente dominante. Em todos os casos analisados, já na primeira linha observou-se que as matrizes não satisfazem a referida condição.

A ausência de dominância diagonal implica que esta condição suficiente para a convergência garantida dos métodos de Jacobi e Gauss-Seidel não é atendida por estas matrizes de Hilbert. Portanto, com base apenas neste critério, não podemos garantir que os métodos de Jacobi e Gauss-Seidel convirjam para a solução dos sistemas lineares envolvendo as matrizes de Hilbert.

A Tabela 3, a seguir, apresenta os resultados de tempo de execução e da norma máxima dos métodos diretos para as matrizes de Hilbert de ordens 5, 9 e 15.

Tabela 3 - Resultados de tempo de execução e da norma máxima dos métodos diretos para as matrizes de Hilbert de ordens 5, 9 e 15.

Método	Ordem da Matriz (n)	Tempo de Execução (s)	Norma Máxima do Erro
Gauss	5	$4,6425 \times 10^{-5}$	$2,591 \times 10^{-12}$
	9	$1,3442 \times 10^{-4}$	$1,6869 \times 10^{-5}$
	15	$5,4487 \times 10^{-4}$	$8,355 \times 10^0$
Pivoteamento Parcial	5	$4,6425 \times 10^{-5}$	$2,591 \times 10^{-12}$
	9	$1,4905 \times 10^{-4}$	$1,6869 \times 10^{-5}$
	15	$5,687 \times 10^{-4}$	$8,313 \times 10^0$
Pivoteamento Parcial com Escala	5	$1,0263 \times 10^{-4}$	$2,591 \times 10^{-12}$
	9	$2,0784 \times 10^{-4}$	$1,6869 \times 10^{-5}$
	15	$6,21398 \times 10^{-4}$	$8,313 \times 10^0$
Pivoteamento Completo	5	$2,207 \times 10^{-4}$	$2,591 \times 10^{-12}$
	9	$2,2359 \times 10^{-4}$	$1,6869 \times 10^{-5}$
	15	$3,46278 \times 10^{-3}$	$8,313 \times 10^0$

Os resultados apresentados da Tabela 3, mostram que os métodos de Gauss e suas variações com pivoteamento obtiveram precisão semelhante para todos os casos. Os valores da norma máxima do erro obtidos por esses métodos aumentaram de acordo com a ordem da matriz de Hilbert.

Em relação ao desempenho computacional (tempo de execução), o método de eliminação de Gauss com pivoteamento parcial foi o que obteve o menor tempo na maior parte dos casos (para $n = 5$ e $n = 15$). A exceção ocorreu para $n = 9$, onde o método de Gauss (sem pivoteamento) apresentou um tempo de execução levemente inferior. Dentre as estratégias de pivoteamento, os métodos com pivoteamento com escala e pivoteamento completo foram, consistentemente, os que demandaram maior tempo de execução. Especificamente, o método com pivoteamento com escala registrou o pior desempenho (maior tempo) para as ordens $n = 5$ e $n = 9$, enquanto o método com pivoteamento completo foi o mais lento para $n = 15$.

Os resultados de desempenho computacional dos métodos de eliminação de Gauss estão de acordo com o esperado: os métodos com pivoteamento completo e com escala são os que possuem maior número de operações (devido às buscas mais extensas por pivôs ou aos cálculos de fatores de escala, respectivamente), o que resulta em maior tempo de execução.

As Tabelas 4, 5 e 6 a seguir apresentam os vetores solução x , obtidos por diferentes métodos de eliminação de Gauss, para as matrizes de Hilbert de ordens $n = 5$, $n = 9$ e $n = 15$.

Tabela 4 - Vetor solução x obtido por diferentes métodos de eliminação de Gauss para a matriz de Hilbert de ordem $n = 5$.

	Gauss	Pivoteamento Parcial	Pivoteamento Parcial com Escala	Pivoteamento Completo
x_1	1,0	1,0	1,0	1,0
x_2	1,0	1,0	1,0	1,0
x_3	1,0	1,0	1,0	1,0
x_4	1,0	1,0	1,0	1,0
x_5	1,0	1,0	1,0	1,0

Tabela 5 - Vetor solução x obtido por diferentes métodos de eliminação de Gauss para a matriz de Hilbert de ordem $n = 9$.

	Gauss	Pivoteamento Parcial	Pivoteamento Parcial com Escala	Pivoteamento Completo
x_1	1,0	1,0	1,0	1,0
x_2	1,0	1,0	1,0	1,0
x_3	1,0	1,0	1,0	1,0
x_4	1,000002	1,000002	1,000002	1,000002
x_5	0,999993	0,999993	0,999993	0,999993
x_6	1,000015	1,000015	1,000015	1,000015
x_7	0,999983	0,999983	0,999983	0,999983
x_8	1,000010	1,000010	1,000010	1,000010
x_9	0,999997	0,999997	0,999997	0,999997

Tabela 6 - Vetor solução x obtido por diferentes métodos de eliminação de Gauss para a matriz de Hilbert de ordem $n = 15$.

	Gauss	Pivoteamento Parcial	Pivoteamento Parcial com Escala	Pivoteamento Completo
x_1	1,0	1,0	1,0	1,0
x_2	1,000002	1,000002	1,000002	1,000002
x_3	0,999995	0,999996	0,999996	0,999996
x_4	0,998640	0,998634	0,998634	0,998634
x_5	1,025668	1,025663	1,025663	1,025663
x_6	0,781933	0,781922	0,781922	0,781922
x_7	2,066841	2,066961	2,066961	2,066961
x_8	– 2,279037	– 2,279032	– 2,279032	– 2,279032
x_9	7,532393	7,532387	7,532387	7,532387
x_{10}	– 7,355048	– 7,355708	– 7,355708	– 7,355708
x_{11}	7,380667	7,337388	7,337388	7,337388
x_{12}	– 1,129041	– 1,101271	– 1,101271	– 1,101271
x_{13}	0,425749	0,415689	0,415689	0,415689
x_{14}	1,733284	1,734771	1,734771	1,734771
x_{15}	0,817952	0,817999	0,817999	0,817999

Observando os resultados das Tabelas 4, 5 e 6, que apresentam os vetores solução para as matrizes de Hilbert de ordens crescentes, nota-se que os coeficientes de menor magnitude exercem um impacto direto no resultado final. Esse comportamento é especialmente evidente na matriz de ordem 15, cujos últimos valores do vetor solução apresentam erros mais significativos. Isso ocorre devido aos erros de aproximação introduzidos por coeficientes extremamente pequenos.

Tabela 7 - Resultados de tempo de execução e da norma máxima dos métodos iterativos com tolerância 10^{-3} para as matrizes de Hilbert de ordens 5, 9 e 15.

Método	Ordem da Matriz (n)	Tempo de Execução (s)	Norma Máxima do Erro
Jacobi	5	n/a	n/a
	9	n/a	n/a
	15	n/a	n/a
Gauss-Seidel	5	$1,1226510 \times 10^{-3}$	$1,12633 \times 10^0$
	9	$5,7542680 \times 10^{-3}$	$1,1561765 \times 10^0$
	15	$1,4885628 \times 10^{-2}$	$1,1376816 \times 10^0$
Sub-Relaxamento (0,8)	5	$1,4233140 \times 10^{-3}$	$1,1361231 \times 10^0$
	9	$2,6703100 \times 10^{-3}$	$1,1839352 \times 10^0$
	15	$1,4652152 \times 10^{-2}$	$1,1303167 \times 10^0$
Sobre-Relaxamento (1,12)	5	$1,7115140 \times 10^{-3}$	$1,1108979 \times 10^0$
	9	$1,2248724 \times 10^{-2}$	$1,1458281 \times 10^0$
	15	$9,9055660 \times 10^{-3}$	$1,147932 \times 10^0$

Analisando os resultados apresentados na Tabela 7, percebe-se que o método de Jacobi não convergiu para nenhuma das ordens da matriz de Hilbert testadas utilizando uma tolerância de 10^{-3} . Por outro lado, os demais métodos iterativos (Gauss-Seidel e Relaxamento SOR) obtiveram valores da norma máxima do erro que foram relativamente próximos entre si para todas as ordens, incluindo $n = 9$ e $n = 15$, situando-se em torno de 1,1 a 1,2. Dos métodos, os que obtiveram melhor e pior precisão para as matrizes de ordens 5 e 9 foram o sobre-relaxamento e o sub-relaxamento, respectivamente. Já para a matriz de ordem 15, o método de sub-relaxamento obteve o melhor resultado e o sobre-relaxamento o pior resultado.

Comparando com os métodos diretos, para as matrizes de Hilbert de ordens $n = 5$ e $n = 9$, os erros dos métodos iterativos que convergiram foram consideravelmente maiores. No entanto, um resultado notável ocorreu para a matriz de Hilbert de ordem $n = 15$: aqui, os valores da norma máxima do erro dos métodos iterativos foram significativamente menores do que os observados para os métodos diretos. Esse comportamento, especialmente para $n = 15$, pode ser atribuído à extrema má condição da matriz de Hilbert de alta ordem, que degrada severamente a precisão dos métodos diretos devido à acumulação dos erros de arredondamento.

Em relação ao desempenho computacional, o método de sobre-relaxamento obteve o pior desempenho para as matrizes de grau $n = 5$ e $n = 9$. Para a matriz de Hilbert de ordem $n = 5$, o método de Gauss-Seidel obteve o melhor desempenho, enquanto para a de ordem $n = 9$, o método de sub-relaxamento obteve o melhor resultado. Já para a matriz de ordem $n = 15$, o método de sobre-relaxamento obteve o melhor desempenho.

Comparando o desempenho, observou-se que os métodos iterativos tiveram um desempenho inferior aos diretos. Especificamente, os tempos de execução para os métodos diretos ficaram entre 10^{-5} s e 10^{-4} s, enquanto os métodos iterativos registraram tempos entre 10^{-3} s e 10^{-2} s, indicando que foram mais lentos. Esse comportamento é, em geral, esperado, pois os métodos iterativos funcionam melhor para sistemas de equações de grande dimensão.

As Figuras 2, 3 e 4, a seguir, apresentam o comportamento da norma máxima do erro dos métodos iterativos durante as iterações para as matrizes de Hilbert de ordens $n = 5$, $n = 9$ e $n = 15$.

Figura 2 - Comportamento da norma máxima do erro dos métodos iterativos com tolerância de 10^{-3} durante as iterações para a matriz de Hilbert de ordem 5.

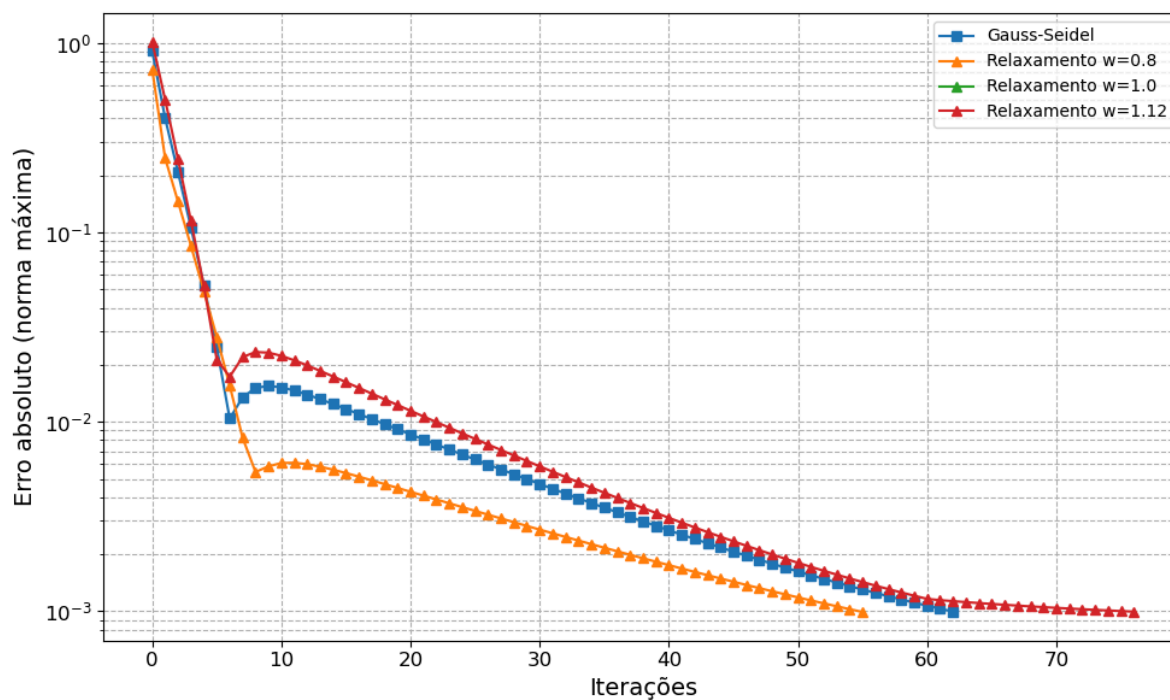


Figura 3 - Comportamento da norma máxima do erro dos métodos iterativos com tolerância de 10^{-3} durante as iterações para a matriz de Hilbert de ordem 9.

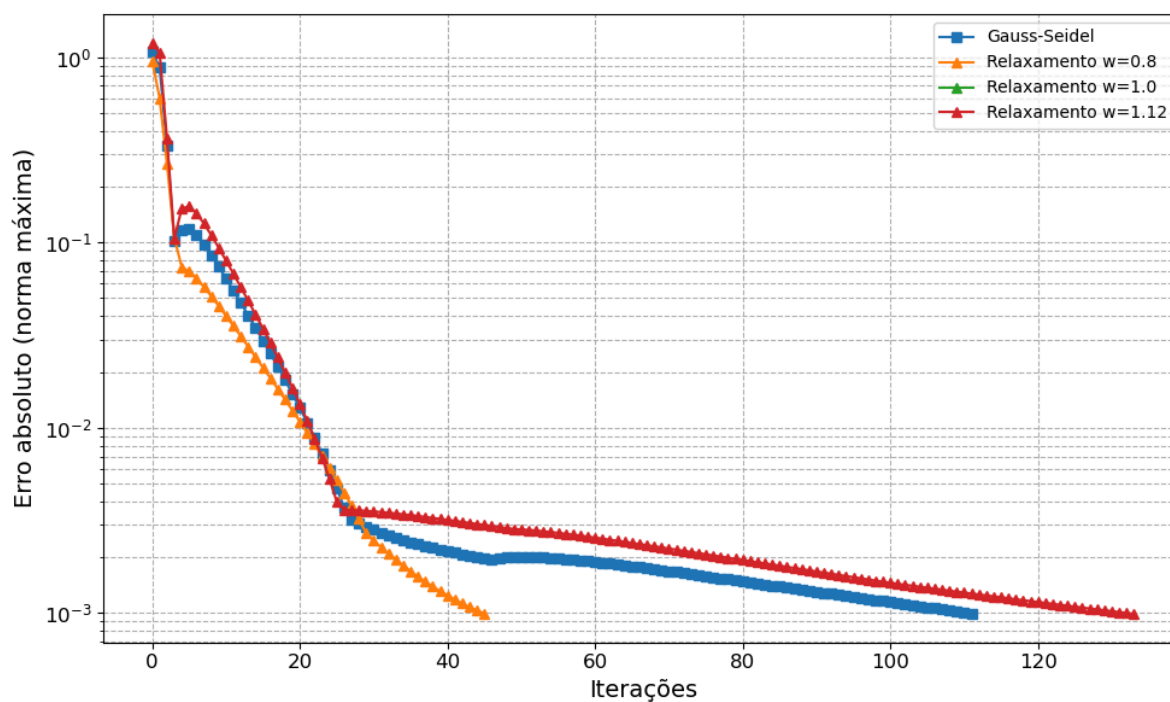
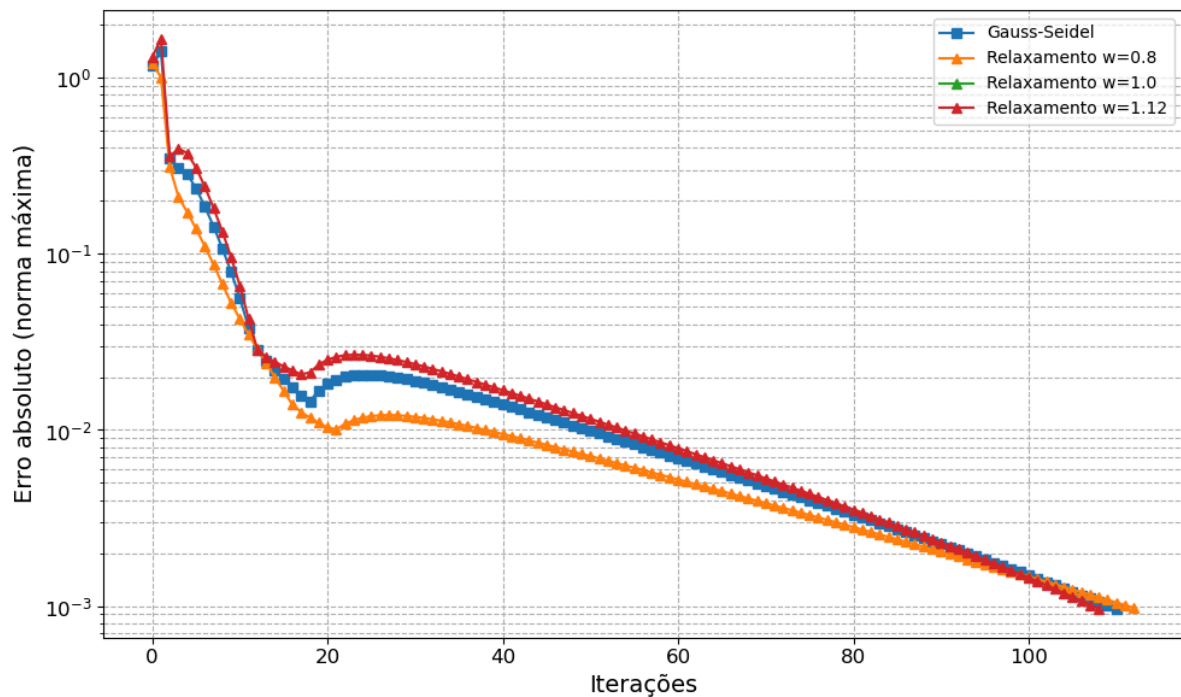


Figura 4 - Comportamento da norma máxima do erro dos métodos iterativos com tolerância de 10^{-3} durante as iterações para a matriz de Hilbert de ordem 15.



Ao analisar os comportamentos da norma máxima do erro apresentados nas Figuras 2, 3 e 4, observou-se que para todos os casos, os erros dos métodos tiveram comportamentos semelhantes para todos os métodos. Tipicamente, o erro apresentava um decréscimo inicial, seguido por um leve aumento, formando um joelho, e, posteriormente, retomava sua trajetória decrescente até a convergência.

Para as matrizes de ordens $n = 5$ e $n = 9$, o método de sub-relaxamento convergiu mais rapidamente, enquanto o de sobre-relaxamento foi o que mais tardou a convergir. Em contrapartida, para a matriz de ordem $n = 15$, o método de sobre-relaxamento demonstrou a convergência mais rápida.

As Tabelas 8, 9 e 10 a seguir apresentam os vetores solução x , obtidos pelos métodos iterativos com tolerância de 10^{-3} , para as matrizes de Hilbert de ordens $n = 5$, $n = 9$ e $n = 15$.

Tabela 8 - Vetor solução x obtido pelos métodos iterativos com tolerância de 10^{-3} para a matriz de Hilbert de ordem $n = 5$.

	Jacobi	Gauss-Seidel	Sub-Relaxamento	Sobre-Relaxamento
x_1	n/a	0,019333	0,021362	0,016722
x_2	n/a	– 0,126330	– 0,136123	– 0,110898
x_3	n/a	0,044628	0,056419	0,02249
x_4	n/a	0,387891	0,382554	0,394228
x_5	n/a	0,660940	0,662808	0,663871

Tabela 9 - Vetor solução x obtido pelos métodos iterativos com tolerância de 10^{-3} para a matriz de Hilbert de ordem $n = 9$.

	Jacobi	Gauss-Seidel	Sub-Relaxamento	Sobre-Relaxamento
x_1	n/a	– 0,019576	– 0,023407	– 0,017369
x_2	n/a	0,138655	0,164653	0,127671
x_3	n/a	0,120492	– 0,128368	– 0,117433
x_4	n/a	– 0,131526	– 0,183935	– 0,145828
x_5	n/a	– 0,156176	– 0,079041	– 0,053523
x_6	n/a	– 0,059190	0,080371	0,085523
x_7	n/a	0,236561	0,244703	0,233918
x_8	n/a	0,380394	0,396313	0,375862
x_9	n/a	0,510904	0,530743	0,505516

Tabela 10 - Vetor solução x obtido pelos métodos iterativos com tolerância de 10^{-3} para a matriz de Hilbert de ordem $n = 15$.

	Jacobi	Gauss-Seidel	Sub-Relaxamento	Sobre-Relaxamento
x_1	n/a	0,007406	0,001091	0,01081
x_2	n/a	– 0,116853	– 0,066493	– 0,147932
x_3	n/a	0,228303	0,146293	0,290927
x_4	n/a	0,069696	0,074528	0,045713
x_5	n/a	– 0,06858	– 0,04548	– 0,079281
x_6	n/a	– 0,132724	– 0,116405	– 0,138826
x_7	n/a	– 0,137682	– 0,130317	– 0,139792
x_8	n/a	– 0,103675	– 0,102304	– 0,103655
x_9	n/a	– 0,046160	– 0,047937	– 0,045073
x_{10}	n/a	0,024395	0,021343	0,025905
x_{11}	n/a	0,101217	0,097966	0,102787
x_{12}	n/a	0,180025	0,177138	0,181464
x_{13}	n/a	0,258176	0,255913	0,25939
x_{14}	n/a	0,334085	0,332537	0,335038
x_{15}	n/a	0,406849	0,406013	0,407538

Observando os resultados das Tabelas 8, 9 e 10, percebe-se que, diferentemente dos valores da solução obtidos pelos métodos diretos, os últimos valores dos vetores solução para todos os métodos iterativos foram os mais próximos dos valores reais. Esse comportamento é devido ao cálculo dos últimos valores das soluções dos métodos de Gauss-Seidel e relaxamento utilizarem os valores dos componentes anteriores já atualizados naquela mesma iteração, incorporando assim a informação mais recente disponível no processo de convergência.

Tabela 11 - Resultados de tempo de execução e da norma máxima dos métodos iterativos com tolerância 10^{-6} para as matrizes de Hilbert de ordens $n = 5$, $n = 9$ e $n = 15$.

Método	Ordem da Matriz (n)	Tempo de Execução (s)	Norma Máxima do Erro
Jacobi	5	n/a	n/a
	9	n/a	n/a
	15	n/a	n/a
Gauss-Seidel	5	$1,1279278 \times 10^0$	$1,0157787 \times 10^0$
	9	$2,1578641 \times 10^0$	$1,1723365 \times 10^0$
	15	$6,4988404 \times 10^0$	$1,1337596 \times 10^0$
Sub-Relaxamento (0,8)	5	$1,4795887 \times 10^0$	$1,0236344 \times 10^0$
	9	$2,3542911 \times 10^0$	$1,1549713 \times 10^0$
	15	$6,1544712 \times 10^0$	$1,1161709 \times 10^0$
Sobre-Relaxamento (1,12)	5	$1,0352957 \times 10^1$	$1,0124067 \times 10^0$
	9	$2,2669329 \times 10^0$	$1,1750819 \times 10^0$
	15	$7,0221492 \times 10^0$	$1,1696325 \times 10^0$

A análise da Tabela 11, apresentada acima, revelou que os métodos iterativos com tolerância de 10^{-6} , com exceção do método de Jacobi, convergiram para todas as matrizes de Hilbert testadas. Ao comparar os resultados dos métodos iterativos com tolerância de 10^{-6} com os obtidos com tolerância de 10^{-3} , observou-se que os métodos alcançaram uma leve melhora na precisão. Contudo, o tempo de execução foi significativamente superior, situando-se entre 1s e 7s.

Ao analisar os valores obtidos pelos métodos iterativos com tolerância de 10^{-6} , observou-se um comportamento semelhante ao obtidos com tolerância de 10^{-3} . Consequentemente, a justificativa para esses valores do vetor é a mesma.

As Figura 5, 6 e 7, a seguir, apresentam o comportamento da norma máxima do erro dos métodos iterativos com tolerância de 10^{-6} durante as iterações para as matrizes de Hilbert de ordens $n = 5$, $n = 9$ e $n = 15$.

Figura 5 - Comportamento da norma máxima do erro dos métodos iterativos com tolerância de 10^{-6} durante as iterações para a matriz de Hilbert de ordem $n = 5$.

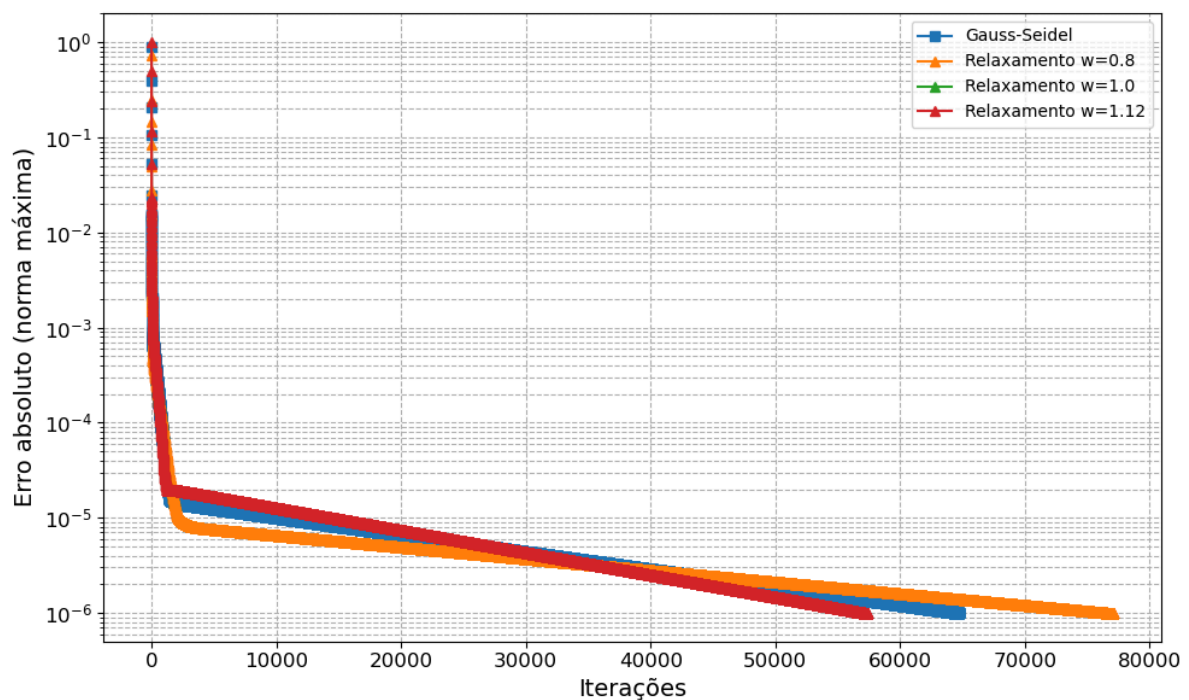


Figura 6 - Comportamento da norma máxima do erro dos métodos iterativos com tolerância de 10^{-6} durante as iterações para a matriz de Hilbert de ordem $n = 9$.

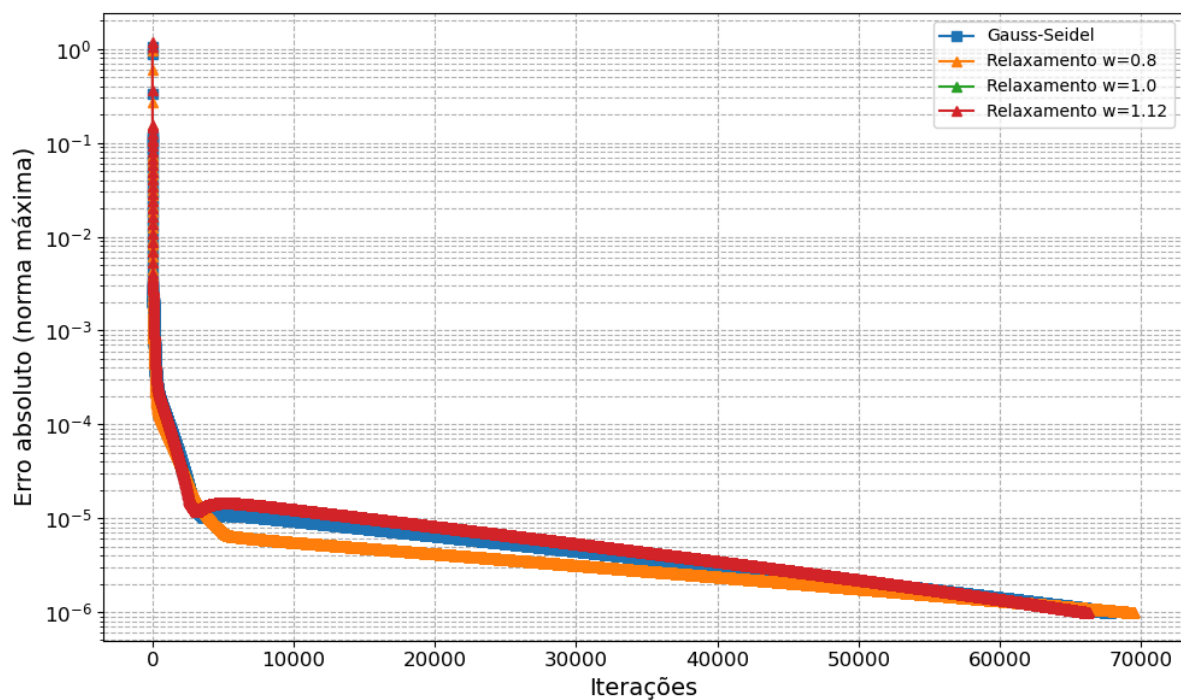
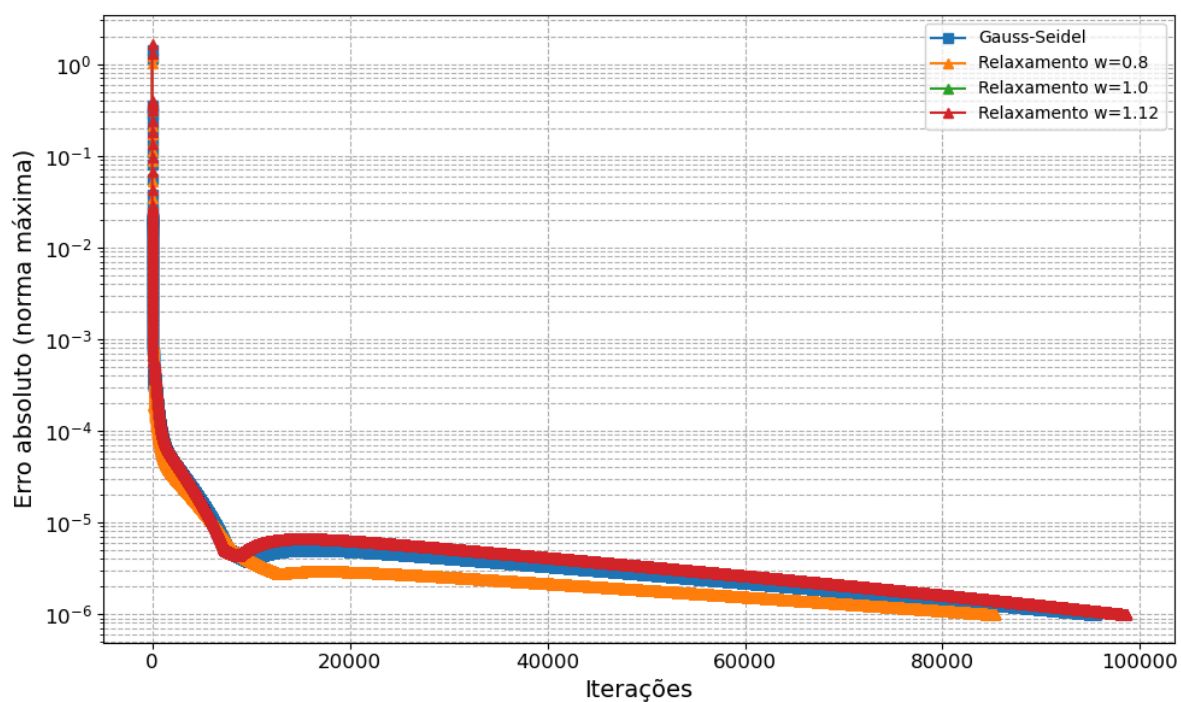


Figura 7 - Comportamento da norma máxima do erro dos métodos iterativos com tolerância de 10^{-6} durante as iterações para a matriz de Hilbert de ordem $n = 15$.



Ao analisar os resultados apresentados nas Figuras 5, 6 e 7, observou-se que houve um grande aumento no número de iterações para atingir a convergência. Para as matrizes de

ordem $n=5$, o aumento no número de iterações foi de 70 para quase 80.000, especificamente para o método de Sobre-Relaxamento. No caso das matrizes de ordem $n=9$, o aumento foi de aproximadamente 150 iterações para quase 70.000 para todos os métodos que convergiram. Por fim, para as matrizes de ordem $n=15$, o aumento foi de cerca de 120 iterações para aproximadamente 100.000 para os métodos de Gauss-Seidel e de Sobre-Relaxamento.

A seguir, são apresentados os resultados obtidos na resolução do exercício 2.

A Tabela 15 exibe a matriz triangular superior U obtida através da fatoração LU da matriz A_1 , utilizada para verificar sua condição de invertibilidade.

Tabela 15 - Matriz triangular superior U obtida através da fatoração LU da matriz A_1 .

10	2	- 1
0	- 5,4	1,7
0	0	5,35185185

Ao analisar a matriz U apresentada na Tabela 13, constatou-se que ela não possui linhas nulas. Isso indica que a matriz original A_1 é invertível. Essa conclusão foi corroborada pelo cálculo do determinante de A_1 , cujo valor obtido foi de - 289, que é diferente de zero.

As Tabelas 16 e 17 a seguir apresentam a matriz inversa obtida pela fatoração LU e a da matriz resultante da multiplicação matricial de A_1 por sua inversa.

Tabela 16 - Matriz inversa da matriz A_1 obtida pela fatoração LU .

$$A_1^{-1} = \begin{bmatrix} 0,11072664 & 0,03806228 & 0,00692042 \\ - 0,05882353 & - 0,17647059 & 0,05882353 \\ - 0,01038062 & 0,02768166 & 0,18685121 \end{bmatrix}$$

Tabela 17 - Matriz resultante da multiplicação matricial de A_1 por sua inversa.

$$A_1 \cdot A_1^{-1} = \begin{bmatrix} 1 & - 1,735 \times 10^{-17} & 0 \\ - 3,12 \times 10^{-17} & 1 & 5,55 \times 10^{-17} \\ - 5,2 \times 10^{-18} & - 2,43 \times 10^{-17} & 1 \end{bmatrix}$$

Observando o resultado da multiplicação matricial $[A_1] \cdot [A_1]^{-1}$, constatou-se que a matriz resultante é muito próxima da matriz identidade. Isso indica que a matriz inversa foi calculada com boa precisão.

Os pequenos desvios em relação à matriz identidade ideal, como elementos fora da diagonal principal que não são exatamente zero, são atribuídos aos erros de arredondamento devido a aritmética de ponto flutuante do computador. Apesar de não serem iguais a zero, os valores dos elementos fora da diagonal principal são muito próximos de zero, confirmando que a matriz inversa foi calculada com uma boa precisão.

A Tabela 18 exibe a matriz triangular superior U obtida através da fatoração LU da matriz A_2 , utilizada para verificar sua condição de invertibilidade.

Tabela 18 - Matriz triangular superior U obtida através da fatoração LU da matriz A_2 .

$$U_{A_2} = \begin{bmatrix} 1 & 4 & 9 & 1,6 \times 10^1 \\ 0 & -7 & -2 & -3,9 \times 10^1 \\ 0 & 0 & 1,143 & 3,4287143 \\ 0 & 0 & 0 & 4,975 \times 10^{-14} \end{bmatrix}$$

Ao analisar a matriz superior U (Tabela 18), observou-se que sua quarta linha é praticamente nula. Embora o último elemento dessa linha não seja estritamente zero, seu valor é um número muito próximo a zero, o que indica que a matriz provavelmente não é invertível. Corroborando essa observação, o determinante da matriz calculado foi de $-1,7053026 \cdot 10^{-13}$, um valor também extremamente próximo de zero, reforçando a conclusão de que a matriz é singular (não invertível).

As Tabelas 19 e 20 a seguir apresentam a matriz inversa obtida pela fatoração LU e a da matriz resultante da multiplicação matricial de A_2 por sua inversa.

Tabela 19 - Matriz inversa da matriz A_2 obtida pela fatora  o LU .

$$A_2^{-1} = \begin{bmatrix} 2,011 \times 10^{13} & -6,032 \times 10^{13} & 6,032 \times 10^{13} & -2,011 \times 10^{13} \\ -6,032 \times 10^{13} & 1,8095 \times 10^{14} & -1,809 \times 10^{14} & 6,032 \times 10^{13} \\ 6,032 \times 10^{13} & -1,809 \times 10^{14} & 1,809 \times 10^{14} & -6,032 \times 10^{13} \\ -2,01 \times 10^{13} & 6,032 \times 10^{13} & -6,032 \times 10^{13} & 2,011 \times 10^{13} \end{bmatrix}$$

Tabela 20 - Matriz resultante da multiplica  o matricial de A_2 por sua inversa.

$$A_2 \cdot A_2^{-1} =$$

$$\begin{bmatrix} 1 \times 10^0 & 0 \times 10^0 & 0 \times 10^0 & 0 \times 10^0 \\ 1,171875 \times 10^{-1} & 1,015625 \times 10^0 & 1,015625 \times 10^{-1} & -1,171875 \times 10^{-1} \\ 9,735 \times 10^{-2} & 8,125 \times 10^{15} & 9,0625 \times 10^{-1} & -9,375 \times 10^{-2} \\ 1,3046875 \times 10^0 & -1,109375 \times 10^0 & 2,5390625 \times 10^0 & -1,796875 \times 10^{-1} \end{bmatrix}$$

Ao analisar a matriz resultante da multiplica  o de A_2 pela matriz obtida ap  s a aplica  o de um algoritmo de invers  o, observou-se que o produto n  o foi a matriz identidade. Esse resultado j   era esperado, dadas as conclus  es das an  lises pr  vias: a verifica  o das linhas da matriz U e o valor calculado para o determinante de A_2 j   indicavam que A_2    uma matriz singular, n  o invert  vel.

5. CONCLUSÃO

Em conclusão, foi possível notar, no exercício 1, que todos os métodos diretos convergem, tanto para uma tolerância de 10^{-3} (considerada alta) quanto para 10^{-6} (considerada baixa). Quanto ao tempo médio para a convergência, os métodos de eliminação de Gauss (sem pivoteamento) e Gauss com pivoteamento parcial com escala alternaram pelo posto do mais rápido. Em alguns testes realizados em máquinas diferentes, o método de eliminação de Gauss com pivoteamento completo foi curiosamente o mais rápido, porém esse fato é exclusivo, já que não se repetiu. Sobre os métodos iterativos, o método de Jacobi não convergiu em nenhum dos testes. Os métodos de Gauss-Seidel e Relaxamento obtiveram valores parecidos de norma máxima do erro com o Sobre-Relaxamento sendo o mais rápido, com $\omega = 1, 2$. O Sub-Relaxamento ($\omega = 0,8$) foi o método mais demorado, porém sua norma máxima do erro foi a menor.

Para o exercício 2, ambas matrizes A_1 e A_2 são mal condicionadas. O determinante da matriz A_1 é igual à -289 . Já o determinante da matriz A_2 é igual à $-1,7053026 \cdot 10^{-13}$, que é um valor muito próximo de 0. Com suas matrizes superiores calculadas, foi possível notar que U_{A_1} não possui valores nulos, ou próximos de 0 em sua diagonal principal, o mesmo não vale para U_{A_2} , pois em sua diagonal principal há o valor $4,975 \times 10^{-14}$, que é muito próximo de 0. Os determinantes de ambas matrizes superiores foram calculados e o valor da determinante de U_{A_2} foi muito próximo de 0, corroborando ainda mais para a suposição de que A_2 não era uma matriz invertível. A_2 é uma matriz não invertível porque suas linhas são linearmente dependentes, visto que os elementos de uma dada linha são múltiplos das outras linhas.

6. APÊNDICE

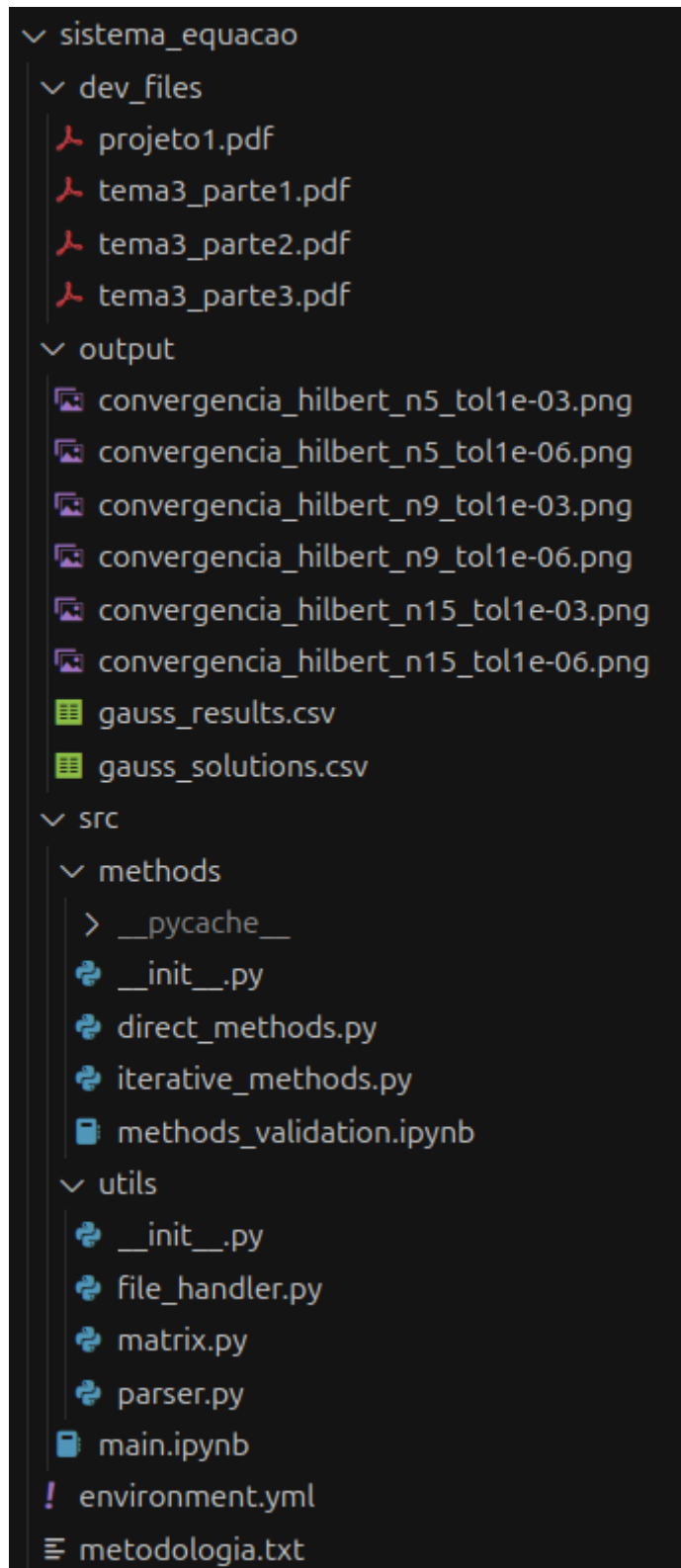
6.1. Link para projeto no GitHub

Os arquivos utilizados para elaboração deste trabalho estão disponíveis através do link <https://github.com/igorssant/metodos_numericos/tree/main/sistema_equacao>.

6.2. Estrutura de pastas e arquivos

A estrutura de pastas está descrita na Figura 8 abaixo.

Figura 8 - Estrutura de pastas e arquivos.



Onde os arquivos contidos na pasta *dev_files* são arquivos disponibilizados em aula pelo professor e foram utilizados para tirar dúvidas. Os arquivos contidos em *output* são as

saídas dos métodos salvos como imagens ou tabelas. Os arquivos e pastas contidas em *src* são os arquivos que compõem os métodos numéricos desenvolvidos. Os arquivos contidos em *methods* são as implementações dos métodos numéricos onde *direct_methods.py* contém as implementações dos métodos diretos e a fatoração *LU*, *interact_methods.py* contém as implementações dos métodos iterativos e *methods_validation.ipynb* é um notebook onde os métodos foram testados e validados utilizando o material de apoio disponível na pasta *dev_files*. A pasta *utils* contém os arquivos de apoio utilizados para auxiliar em cálculos, leitura e escrita, onde *file_handler.py* contém funções para ler e escrever em arquivos de texto, *matrix.py* contém funções para auxiliar a criação e resolução de matrizes e *parser.py* contém algoritmos para transformar funções matemáticas escritas como *string* para objetos do pacote *symPy*, bem como algoritmos para resolver essas funções matemáticas. O arquivo *main.ipynb* é o notebook onde os exercícios foram resolvidos. O arquivo *environment.yml* contém um ambiente virtual em python gerado para a implementação dos arquivos utilizados neste projeto. O arquivo *metodologia.txt* contém um protótipo da metodologia utilizada no projeto.

6.3. Instalando as dependências do projeto

O ambiente do projeto está disponível no arquivo *environment.yml*. Para baixar as dependências, é necessário que o software Anaconda esteja instalado na máquina desejada. O ambiente será gerado automaticamente utilizando o comando: `conda env create --file=/caminho/ate/arquivo/yml/environment.yml`.

O ambiente então será gerado e gerenciado pelo software Anaconda. Uma vez criado o ambiente, os arquivos poderão ser executados sem problemas.